ASSIGNMENT 1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Object-oriented programming principles and C++ constructs (data types, operators, control structures)

Name	S MOHAMED AYDIN
Register Number	192424090
Course	DSA0102
Submission Date	20.10.2026
Compiler	g++ 13.3.0 (Ubuntu 24.04)
Compile command	g++ -std=c++17 -Wall -Wextra file.cpp -o file
GitHub Repository	https://github.com/mohamedaydin006/DSA0102.git

Fill in the bracketed fields before submitting. All programs in this report were compiled with the command above and every output shown is copied from a real run (not typed by hand). Assumptions are stated where the scenario leaves a value open (for example, the concession percentage).

Contents, assumptions and how to reproduce

Task	Topic	Folder / source
1	Object-oriented analysis	(analysis only)
2	Tokens, keywords, identifiers, constants	Task02/constants.cpp
3	Data types and type compatibility	Task03/datatypes.cpp
4	Implicit conversions (predict the output)	Task04/predict.cpp, brace_error.cpp, explicit_version.cpp
5	Operators and precedence (predict the output)	Task05/predict.cpp, expressions.cpp
6	Control structures (complete the code)	Task06/fare.cpp, boundary.cpp
7	Procedural vs object-oriented	Task07/procedural.cpp, oop.cpp
8	Input and output	Task08/io_cin.cpp, io_getline.cpp
9	Implementation and testing	Task09/fare_tests.cpp
10	Expectation vs execution	Task10/experiment.cpp
11	Final recommendation	Task11/config_fares.cpp
12	Day-4 design challenge	Task12/redesign.cpp

Assumptions used throughout

- All money is stored as **integer paise** in `long long` (Rs 1 = 100 paise).
- The fare depends on the **farthest zone travelled**: Inner Rs 10, Middle Rs 20, Outer Rs 30.
- Peak hours are 08:00 to 09:59 and 17:00 to 19:59, i.e. the half-open intervals [08:00, 10:00) and [17:00, 20:00). So 8:00 and 17:00 are peak; 10:00 and 20:00 are off-peak (this fixes the inconsistency in the first gate software).
- The peak surcharge is 20 % of the base fare, computed with integer arithmetic.
- The scenario does not give the concession rate. I assume **30 % off** for students and senior citizens, applied after the surcharge and rounded half up. It is one constant, `CONCESSION_PERCENT`.
- A journey is allowed only if the balance remaining after the fare is at least Rs 50 (5000 paise).
- For Task 12: on weekends the 10 % discount **replaces** the peak surcharge (they never combine); the monthly pass covers 30 days; pass journeys deduct nothing and the Rs 50 minimum is not enforced for them (nothing is deducted).

Folder layout of the GitHub repository

```
DSA01-<RegisterNumber>-Assignment1/
  README.md
  Task01/ ... Task12/        (one folder per task: .cpp, input files, output screenshots)
```

Commit plan (at least one commit per task)

```
git init
git add Task02 && git commit -m "Task 2: constants, enum class, macro comparison"
git add Task03 && git commit -m "Task 3: data types, sizeof, precision and overflow"
... (one commit per task, Task 1 = README analysis) ...
git remote add origin https://github.com/<you>/DSA01-<RegisterNumber>-Assignment1.git
git push -u origin main
```

TASK 1 - Understand the Problem and Identify the Objects (4 Marks)

1.1 Objects, data and behaviour

Object	Data (attributes)	Behaviour (member functions)	Why it is an object
SmartCard	balance (paise, private), card id, optional Pass	recharge(), canTravel(fare), deduct(fare), balance()	Has state that must stay valid (never negative, Rs 50 kept) so the data must be guarded by its own functions.
Passenger	name, category (Normal / Student / Senior)	concessionPercent()	Category decides the concession; changes independently of the card.
Journey	entry zone, exit zone, hour, minute, weekend flag	farthestZone(), isPeak()	One event processed by the gate; carries the inputs of the fare rule.
Zone	Inner, Middle, Outer (enum class)	-	Closed set of values; wrong values must be impossible to write by accident.
FareTable	base fare per zone, surcharge %, concession %	baseFare(zone), fare(journey, passenger)	All fare numbers live here, so a revision is made in one place.
Gate	FareTable, running totals	process(card, journey)	Coordinates the objects for each tap.

1.2 Procedural style versus object-oriented style

In a **procedural** version the data are separate variables (`balance`, `zone`, `peak`) and free functions (`fare()`, `canTravel()`) take them as parameters. Any statement in the program can change `balance`. In the **object-oriented** version the data and the rules that protect it are bundled in `SmartCard`; the only way to reduce the balance is `deduct()`, which refuses a journey that would break the rule. Task 7 demonstrates this with running code (the procedural version ends with a balance of -2999 paise; the class version cannot).

1.3 The four principles of OOP in this system

Principle	Where it appears here
Encapsulation	SmartCard keeps *balance_* private; only recharge()/deduct() change it, so the invariant 'balance never below Rs 50 after a journey' cannot be bypassed.
Abstraction	The gate calls canTravel(fare) and does not need to know how the Rs 50 rule is computed; callers see what, not how.
Inheritance	Student and SeniorCitizen can derive from Passenger (or ConcessionRule) and reuse its data, changing only the concession.
Polymorphism	A virtual concessionPercent() lets the gate call the same function on any Passenger and get Normal 0 %, Student 30 %, Senior 30 %.

1.4 Rules every correct journey must satisfy

- R1: balance after deduction >= 5000 paise (Rs 50).
- R2: the balance is never negative (a stronger consequence of R1 for valid cards).
- R3: fare = base(farthest zone), plus 20 % when the time is in [08:00,10:00) or [17:00,20:00).
- R4: all money is integer paise; no floating point in fare or balance, so every fare is exact.
- R5: an unknown zone, a negative or zero recharge, or non-numeric input is rejected without changing the card.
- R6: a fare revision must require editing one place only.

1.5 Alternative concept - purely procedural style

Solution if procedural: global variables `balance`, `zone`, `peak` and free functions `fare(zone, peak)`, `canTravel(balance, f)`. **Time per journey:** one switch, one multiply, one divide, one compare: O(1) in both designs. **Memory per card:** one long long (8 bytes) in both: O(1); the class adds no data (no virtual functions in SmartCard). **Code to change when one rule changes:** in the procedural version every function and every place that touches the global balance may need review (the whole program, O(number of code locations)); in the object-oriented version the rule lives in canTravel() and the constants, so one function plus one constant changes.

Assumptions stated: see the list on the previous page (concession 30 %, farthest-zone fare, half-open peak intervals).

TASK 2 - Use Tokens, Keywords, Identifiers and Constants Correctly (3 Marks)

2.1 The five kinds of tokens (examples taken from Task06/fare.cpp)

Token kind	Examples in the program
Keywords	enum, class, constexpr, int, long, bool, switch, case, default, return, if, else, for, const, struct
Identifiers	Zone, Inner, BASE_PAISE, PEAK_SURCHARGE_PERCENT, fare, canTravel, balance, trips
Constants (literals)	1000, 2000, 3000, 20, 5000, 100, 0, true, false, '.', '0', "Denied (low balance)"
Operators	=, +=, -=, *, /, %, >=, -, <<, ::, ++
Punctuators	{ } ( ) [ ] ; , :

2.2 #define vs const vs constexpr vs enum class

Feature	#define	const	constexpr	enum class
Handled by	Preprocessor (text)	Compiler	Compiler	Compiler
Type checked	No	Yes	Yes	Yes (own type)
Scope	None - file-wide from the #define; ignores namespaces and blocks	Block / namespace	Block / namespace	Scoped (Zone::Inner)
Usable in array size / static_assert	Yes	Only if initialised by a constant expression	Always (guaranteed)	Yes
Debugger shows name	No	Yes	Yes	Yes
Typical use	include guards, conditional compilation	run-time read-only values	**fare table, percentages, limits**	**zone names**

Justified choice: `constexpr` for the fare table, surcharge percent and minimum balance (typed, scoped, compile-time guaranteed, a revision is one line); `enum class Zone` for zones; no `#define` at all. The macro below is the evidence of why.

2.3 enum vs enum class

A plain `enum` leaks its names into the surrounding scope (`Inner` could clash with another `Inner`) and converts silently to int, so `int z = Green;` compiles and `if (zone == 7)` is accepted. With `enum class` the names are scoped (`Zone::Inner`), there is no implicit conversion to or from int, so `Zone w = 1;` and `int z = Zone::Inner;` are compile errors. The compiler can also warn (-Wswitch) when a switch forgets an enumerator.

2.4 Naming rules and conventions

- **Rules (enforced by the compiler):** letters, digits and underscore only; must not start with a digit; case-sensitive; cannot be a keyword; avoid names with a double underscore or a leading underscore + capital (reserved).
- **Conventions used:** constants in UPPER_SNAKE_CASE (*BASE_PAISE*), types in PascalCase (*SmartCard*, *Zone*), functions and variables in camelCase (*canTravel*, *balance*), private data members end with an underscore (*balance_*), names carry the unit (*_PAISE*, *_PERCENT*).

2.5 Alternative concept - #define macros

Source: Task02/constants.cpp

```
// Task 2: tokens, constants, enum class vs macros
#include <iostream>

// ---- the recommended single place for all fare constants ----
enum class Zone { Inner, Middle, Outer };            // scoped, type-safe
constexpr int       BASE_PAISE[] = {1000, 2000, 3000};
constexpr int       PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

// ---- the macro alternative (what NOT to do) ----
#define OLD_PEAK_PERCENT 10 + 10      // intended 20, but no parentheses
#define OLD_INNER 1000
#define OLD_MIDDLE 2000
```

```
int main()
{
    int base = BASE_PAISE[static_cast<int>(Zone::Middle)];
    std::cout << "constexpr surcharge : " << base * PEAK_SURCHARGE_PERCENT / 100 << '\n';
    std::cout << "macro surcharge      : " << base * OLD_PEAK_PERCENT / 100
              << "  (expands to base * 10 + 10 / 100)\n";

    // enum (unscoped) vs enum class
    enum Colour { Red, Green };       // unscoped: converts to int silently
    int c = Green;                    // compiles, no cast needed
    std::cout << "unscoped enum -> int = " << c << '\n';
    // int z = Zone::Inner;           // ERROR for enum class: no implicit int conversion
    // Zone  w = 1;                   // ERROR: int is not a Zone
    std::cout << "sizeof(Zone) = " << sizeof(Zone) << ", sizeof(BASE_PAISE) = " << sizeof(BASE_PAISE) << '\n';
}
```

Actual output

```
constexpr surcharge : 400
macro surcharge     : 20000  (expands to base * 10 + 10 / 100)
unscoped enum -> int = 1
sizeof(Zone) = 4, sizeof(BASE_PAISE) = 12
```

If fares were macros: `#define OLD_PEAK_PERCENT 10 + 10` expands textually, so `base * OLD_PEAK_PERCENT / 100` becomes `base * 10 + 10 / 100` = 20000 instead of 400: a silent 50x error that the constexpr version cannot make. **Type checking:** none for macros (the number has no type); constexpr has a type. **Scope:** a macro ignores namespaces and blocks and can replace identifiers anywhere below it; constexpr and enum class are scoped. **Memory:** a macro occupies no storage (the number is pasted into the instructions); a constexpr scalar usually occupies none either (the compiler folds it in); the constexpr array BASE_PAISE takes 12 bytes only if it is indexed at run time (shown above: sizeof = 12). **Running time:** not affected; after optimisation the code for both is the same.

TASK 3 - Choose Data Types and Check Type Compatibility (4 Marks)

Evidence: Task03/datatypes.cpp output on this system (g++ 13.3, 64-bit Linux)

```
---- sizeof and ranges on this system ----
bool           1 bytes
char           1 bytes  [-128, 127]
int            4 bytes  [-2147483648, 2147483647]
unsigned       4 bytes  [0, 4294967295]
long long      8 bytes  [-9223372036854775808, 9223372036854775807]
float          4 bytes  max 3.40282e+38, ~6 digits
double         8 bytes  max 1.79769e+308, ~15 digits

---- mixed expression: int + double + bool ----
i + d + b = 4.5  (type size 8 bytes => double)

---- precision problem: money as double rupees ----
0.1 added 10 times = 0.99999999999999988898
(sum == 1.0) is false

rounding error after n additions of Rs 0.10 (exact = n/10):
n=      1000  error=1.4068746168e-12
n=     10000  error=1.5882051230e-10
n=    100000  error=1.8848368200e-08
n=   1000000  error=1.3328826753e-06
n=  10000000  error=1.6102462541e-04

---- overflow problem: int total for the network ----
exact total (long long) = 6000000000 paise
INT_MAX                 = 2147483647 paise
stored in int           = 1705032704 paise  (WRONG)
int is enough for 20,000 journeys? true
int is enough for 500,000 journeys? true
int is enough for 3,000,000 journeys? false
```

3.1 Data-type table (sizes and ranges measured with sizeof and)

Value	Type chosen	sizeof	Range	Justification
Fare (paise)	long long	8	-9.22e18 .. 9.22e18	Exact integer; same type as balance and totals so no conversion; int would fit one fare but not sums.
Card balance (paise)	long long	8	same	Exact; signed so a bug that goes below 0 is visible (-2999) rather than wrapping to 4 billion.
Time: hour, minute	int	4	-2.1e9 .. 2.1e9	Natural machine word; comparisons and minutes = h*60+m are fast; range 0-23, 0-59 is checked by the program.
Zone number	enum class Zone (underlying int)	4	3 named values	Type-safe; cannot mix with plain ints or other enums. A char would also work but saves nothing.
Journey count	long long (unsigned also fits)	8	up to 9.22e18	3,000,000 fits in int, but counts are multiplied by paise, so long long avoids overflow in products.
Percent (surcharge, concession)	int	4	0..100	Whole numbers; used only in integer arithmetic.
Peak flag, allowed flag	bool	1	false/true	Exactly two states.
Loop temporaries	auto	deduced	-	Avoids repeating long types (e.g. in range-for); type still fixed at compile time.
Rupee display	none stored - derived	-	-	paise / 100 and paise % 100 are printed; money is never stored as double.

char (1 byte, -128..127 here), unsigned (4 bytes, 0..4,294,967,295), float (4 bytes, ~6 digits) and double (8 bytes, ~15 digits) were also measured above. float and double are not used for money.

3.2 Why money must be integer paise, not double rupees

A double stores numbers in binary. Values like 0.1, 25.2 and 8.4 have no exact binary form, so each one is stored as a close neighbour. The evidence above: adding 0.1 ten times gives 0.99999999999999988898 and `sum == 1.0` is **false**. Every addition rounds again, so the error accumulates. Integers are exact: 2520 paise is always exactly 2520, so totals, comparisons (`balance - fare >= 5000`) and equality tests are reliable, and the balance rule can never fail because of 4999.9999999.

3.3 Type compatibility rules and mixed expressions

- **Usual arithmetic conversions:** in a binary operation both operands are brought to a common type. bool and char are first promoted to int; then if either is double the other becomes double; otherwise long long outranks int; and for signed/unsigned of the same rank the signed value becomes unsigned.
- **Evidence:** `int i=3; double d=0.5; bool b=true;` then `i + d + b`: i becomes 3.0, (3.0+0.5)=3.5, b becomes 1 then 1.0, result 4.5 and the result type has 8 bytes (double).
- **Assignment** converts to the target type: double to int truncates the fraction; int to bool is false only for 0; a wider integer to a narrower one keeps the low bits (wrap-around).

3.4 Ranges from : is int enough for a day's collection?

The mixed fare pattern used in Task 10 averages 1870 paise per journey. A day's collection is: 20,000 journeys = 37,400,340 paise (fits in int, limit 2,147,483,647); 500,000 journeys = 935,000,340 paise (fits, but already 43 % of the limit); 3,000,000 journeys = 5,610,000,000 paise, which is **2.6 times** INT_MAX. **int is therefore not enough for the network total** (it overflows at about 1.15 million journeys at this average fare). The output above shows the wrapped value 1,705,032,704 for the 2,000-paise case: exactly the kind of silent corruption that must be avoided. **long long** holds up to 9.22e18 paise.

3.5 Alternative concept - double rupees instead of long long paise

Solution if double: `double fareRs = 36.0; double balanceRs = 100.0;` and compare `balanceRs - fareRs >= 50.0`. **Time per addition:** O(1) for both (one machine instruction; measured in Task 10: about 0.55 ns for long long and 0.80 ns for double with -O2). **Space per value:** 8 bytes for both. **Rounding error when n fares are added** (measured above, adding Rs 0.10 n times): n = 1,000: 1.4e-12; 10,000: 1.6e-10; 100,000: 1.9e-8; 1,000,000: 1.3e-6; 10,000,000: 1.6e-4. Each addition adds a rounding error proportional to the size of the running sum, and the running sum itself grows with n, so the error grows faster than linearly (about 70-120 times per 10-times increase of n here, roughly n squared in the worst case). With long long paise the error is exactly **0** for every n.

3.6 Precision / overflow problems found and their resolution

Problem	Evidence	Resolution
Precision (double)	0.1 x 10 is 0.9999999999999988898, (sum == 1.0) false	Store paise in long long; convert to rupees only when printing
Overflow (int)	6,000,000,000 stored in int = 1,705,032,704 (WRONG)	Use long long for any accumulated total or product

TASK 4 - Control Implicit Conversions (4 Marks)

4.1 Predicted output (written BEFORE running)

```
3 3.5
C 68
1 true
s >= u
```

Reasoning for each line is given below. Important: this is the section the assignment asks you to write before running. If you submit your own version, write your prediction first in your own words and only then run the program.

Program (Task04/predict.cpp, given)

```
#include <iostream>

int main()
{
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';

    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';

    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

Actual output

```
3 3.5
C 68
1 true
s >= u
```

Compiler warning with -Wall -Wextra

```
Task04/predict.cpp: In function 'int main()':
Task04/predict.cpp:18:21: warning: comparison of integer expressions of different signedness: 'int' and 'unsigned int' [-Wsign-compare]
   18 |     std::cout << (s < u ? "s < u" : "s >= u") << '\n';
      |                   ~~^~~
```

4.2 Prediction vs actual: line-by-line explanation

Difference between my prediction and the actual output: none - all four lines matched.

Output line	Conversion that took place
3 3.5	**Integer division:** *a / b* is int / int = 3 (fraction discarded) and only then is 3 converted **int to double** (3.0, printed as 3). In *d*, static_cast<double>(a) is done first, so it is double / int: b is converted to 2.0 and the result 3.5 is kept.
C 68	'A' + 2: char 'A' (65) is promoted to **int**, the sum 67 is converted back **int to char** (narrowing) to give 'C'. Printing *ch* prints the character; *ch + 1* promotes ch to int so it is the int 68, printed as a number.
1 true	**int to bool:** any non-zero value (5) becomes true, stored as 1. Without boolalpha it prints 1; with std::boolalpha it prints true.
s >= u	**Signed to unsigned:** int -1 is converted to unsigned int 4,294,967,295 for the comparison, so s < u is false. This is the dangerous conversion, and gcc warns (-Wsign-compare).

4.3 Narrowing conversions and brace initialisation

Plain initialisation `int y = 3.7;` compiles and silently stores 3. Brace initialisation forbids any conversion that may lose information, so `int x{3.7};` is rejected at compile time. Evidence from compiling Task04/brace_error.cpp:

```
Task04/brace_error.cpp: In function 'int main()':
Task04/brace_error.cpp:4:11: error: narrowing conversion of '3.7000000000000002e+0' from 'double' to 'int' [-Wnarrowing]
    4 |     int x{3.7};          // error: narrowing conversion of 3.7 from double to int
      |           ^~~
```

4.4 static_cast and when an explicit cast is needed

An explicit cast is needed when the programmer *wants* a conversion the compiler would not do the way they intend: `static_cast<double>(a) / b` forces real division; `static_cast<int>(ch)` prints a char's code;

and a deliberate narrowing such as `static_cast<int>(3.7)` documents the intent. static_cast is checked at compile time (it refuses unrelated types, unlike a C-style cast) and is easy to search for.

4.5 Alternative concept - every conversion explicit, brace initialisation everywhere

Source: Task04/explicit_version.cpp

```
// Same program as predict.cpp with every conversion written explicitly
#include <iostream>

int main()
{
    int a{7}, b{2};
    double c{static_cast<double>(a / b)};
    double d{static_cast<double>(a) / static_cast<double>(b)};
    std::cout << c << ' ' << d << '\n';

    char ch{static_cast<char>('A' + 2)};
    std::cout << ch << ' ' << static_cast<int>(ch) + 1 << '\n';

    bool flag{5 != 0};
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';

    unsigned int u{1};
    int s{-1};
    std::cout << (static_cast<long long>(s) < static_cast<long long>(u) ? "s < u" : "s >= u") << '\n';
}
```

Actual output

```
3 3.5
C 68
1 true
s < u
```

Running time and memory: unchanged. The conversions already happened implicitly; static_cast generates the same instructions, so the cost is the same (O(1) per conversion) and no extra memory is used. **Errors caught at compile time:** narrowing (`int x{3.7}` is an error), int to enum class, silent double to int losses; each cast also lets -Wconversion stay quiet only where the programmer chose it. **Errors still found only at run time:** overflow, division by zero, a value that is out of range. A cast can even *hide* a bug. **Note the last line:** the explicit version prints **s < u** because I deliberately converted both to long long, so -1 < 1 is true; this is the behaviour the programmer meant, while the implicit version gave the surprising **s >= u**. Explicit conversions therefore make the intended meaning visible and can change the result.

TASK 5 - Write Correct Expressions with the Right Precedence (4 Marks)

5.1 Predicted output (written BEFORE running)

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Program (Task05/predict.cpp, given)

```
#include <iostream>

int main()
{
    int x = 5, y = 3;
    std::cout << x + y * 2 << ' ' << (x + y) * 2 << '\n';

    int z = x++ + ++y;
    std::cout << z << ' ' << x << ' ' << y << '\n';

    std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';
    std::cout << 10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

    int k = 0;
    for (int i = 0; i < 10; i += 3) k += i;
    std::cout << k << '\n';
}
```

Actual output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Difference between prediction and actual output: none.

5.2 Operator-by-operator explanation

Line	Precedence / associativity applied	Value
x + y * 2	***** (multiplicative) binds tighter than **+** (additive): y*2 = 6, then 5+6.	11
(x + y) * 2	Parentheses first: 8, then * 2.	16
int z = x++ + ++y;	Tokenised as *(x++) + (++y)* (postfix and prefix bind tighter than binary +; the lexer takes the longest token **++**). x++ yields the old value 5 and then x = 6; ++y makes y = 4 and yields 4. Binary + is left-to-right: 5 + 4 = 9, then **=** (right-to-left) assigns it to z. x and y are different variables, so there is no undefined behaviour.	z=9, x=6, y=4
(x > y ? x : y)	**>** (relational) is higher than **?:**, which is right-associative and lower than relational/logical but higher than assignment. Parenthesised because **<<** binds tighter than ?: . 6 > 4 is true, so x.	6
x % 4 * 2	**%** and ***** have the same precedence and associate left-to-right: (6 % 4) * 2 = 2 * 2.	4
10 / 3, 10 % 3	Integer division truncates: 3 remainder 1.	3 1
-10 / 3, -10 % 3	Unary minus applies first. Since C++11 division truncates **toward zero**: -3, and the remainder takes the sign of the dividend so that (a/b)*b + a%b == a: -3*3 + (-1) = -10.	-3 -1
for (...; i += 3) k += i;	**+=** is compound assignment (right-to-left, lowest precedence). i takes 0, 3, 6, 9 and k = 0+3+6+9.	18

5.3 Expressions for the peak check, surcharge and concession

Source: Task05/expressions.cpp

```
// Task 5: the peak check, the surcharge and the concession as correct expressions
#include <iostream>

bool isPeakHour(int hour)
{
    return (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20);
}

// lookup-table alternative
constexpr bool PEAK_TABLE[24] = {
    false,false,false,false,false,false,false,false,     // 0-7
```

```
    true, true,                                          // 8,9
    false,false,false,false,false,false,false,           // 10-16
    true, true, true,                                    // 17,18,19
    false,false,false,false};                           // 20-23

int main()
{
    // surcharge: integer arithmetic, one rounding, no double
    long long base = 3000;
    long long withSurcharge = base + base * 20 / 100;         // * and / bind tighter than +
    // concession: 30 % off, round half up, integer only
    long long concession = (withSurcharge * (100 - 30) + 50) / 100;
    // conditional operator: right-assoc, lower than ||, higher than =
    long long finalFare = isPeakHour(9) ? withSurcharge : base;

    std::cout << "surcharge fare = " << withSurcharge << " paise\n";
    std::cout << "concession fare = " << concession << " paise\n";
    std::cout << "conditional fare = " << finalFare << " paise\n";

    for (int h : {7, 8, 9, 10, 16, 17, 19, 20})
        std::cout << "hour " << h << ": expression=" << isPeakHour(h)
                  << " table=" << PEAK_TABLE[h] << '\n';

    int i = 5;
    i = i++;                        // undefined before C++17 / unsequenced warning; i stays 5 in C++17
    std::cout << "i = i++ gives i = " << i << " (the increment is lost - a mistake)\n";
}
```

Actual output

```
surcharge fare = 3600 paise
concession fare = 2520 paise
conditional fare = 3600 paise
hour 7: expression=0 table=0
hour 8: expression=1 table=1
hour 9: expression=1 table=1
hour 10: expression=0 table=0
hour 16: expression=0 table=0
hour 17: expression=1 table=1
hour 19: expression=1 table=1
hour 20: expression=0 table=0
i = i++ gives i = 5 (the increment is lost - a mistake)
```

gcc warning for the deliberate mistake i = i++

```
Task05/expressions.cpp: In function 'int main()':
Task05/expressions.cpp:36:7: warning: operation on 'i' may be undefined [-Wsequence-point]
   36 |     i = i++;                        // undefined before C++17 / unsequenced warning; i stays 5 in C++17
      |     ~~^~~~
Task05/expressions.cpp:36:7: warning: operation on 'i' may be undefined [-Wsequence-point]
```

- **Peak check:** `(hour >= 8 && hour < 10) || (hour >= 17 && hour < 20)`. Relational operators bind tighter than &&, and && tighter than ||, so the version without parentheses (hour >= 8 && hour < 10 || hour >= 17 && hour < 20) means exactly the same, but a reader has to remember that rule. The parenthesised form states the intent (and avoids -Wparentheses warnings), so it is the better choice.
- **Surcharge:** `base + base * 20 / 100`: * and / (left-to-right) happen before +, integer arithmetic, no rounding error for multiples of 5 paise.
- **Concession:** `(fare * (100 - 30) + 50) / 100`: parentheses force the subtraction first; +50 rounds half up in integer arithmetic.
- **Conditional and compound assignment:** `fare = isPeak ? withSurcharge : base;` ?: is evaluated before =. `balance -= f` is shorthand for balance = balance - f.

5.4 x++ versus ++x, and why i = i++ is a mistake

`x++` (post-increment) yields the *old* value and then increments; `++x` (pre-increment) increments and yields the *new* value. In `i = i++;` the old value of i is copied back into i after the increment, so the increment is lost (the program above prints i = 5, not 6). Before C++17 the statement was undefined behaviour (two unsequenced modifications of i) and gcc still warns with -Wsequence-point. The correct statements are `i++;` or `i = i + 1;`.

5.5 Alternative concept - lookup table bool peak[24]

Solution: `constexpr bool PEAK_TABLE[24] = {...}` with true at indices 8, 9, 17, 18, 19; the check is `PEAK_TABLE[hour]` (shown above; both versions agree for hours 7, 8, 9, 10, 16, 17, 19, 20). **Time:** O(1) for both; the expression does up to 4 comparisons and 1-3 logical tests, the table one index operation, so the table is fewer operations. **Extra space:** 24 bytes (one bool each). **Trade-off:** the table works on whole hours only; the minute-level rule used in Tasks 6 and 9 (minutes since midnight) needs a 1440-entry table or the expression. Changing the peak hours means editing array positions, which is easier to get wrong than editing two bounds; an

out-of-range hour reads outside the array unless it is checked first.

TASK 6 - Design the Control Structures (4 Marks)

6.1 Decision logic: if-else, switch, conditional operator

- **switch on the zone:** the zone is a discrete enum with a handful of values and one action per value, so a switch is clearer than a chain of if-else, the compiler can build a jump table, and -Wswitch warns when an enumerator is missing.
- **if-else:** for conditions that are ranges or combinations (the Rs 50 balance rule, the peak interval, concession yes/no).
- **Conditional operator ?: :** for a short choice between two values, e.g. `peak ? base + base*20/100 : base`. Not for long branches or nesting.
- **What happens when no case matches:** without a default, nothing in the switch runs and control continues after it, so a variable that was meant to be set stays uninitialised (and a function without a return is undefined behaviour). Therefore a default must be written: here it returns -1 for an unknown zone and canTravel() rejects negative fares.

6.2 Loops: for, range-based for, while

- **Range-based for** (`for (const Trip& t : trips)`) is best for processing every journey: no index, no off-by-one error, and const& avoids copies.
- **Counting for** is needed only when the index is used (e.g. printing journey numbers).
- **while** is right when the number of journeys is unknown, e.g. `while (std::cin >> zone >> hour)` reading until the end of input. All three are O(n) for n journeys.
- **break** leaves the loop (or ends a switch case; forgetting it causes fall-through into the next case); **continue** skips the rest of this iteration, ideal for skipping an invalid journey:

```
for (const Trip& t : trips) {
    long long f = fare(t.zone, t.peak);
    if (f < 0) { std::cout << "Invalid journey skipped\n"; continue; }   // continue: next journey
    if (!canTravel(balance, f)) { std::cout << "Card blocked, stopping\n"; break; } // break: stop all
    balance -= f;
}
```

6.3 Pseudocode for the fare computation

The SIMATS Engineering Standard for Algorithmic Pseudocode is a document I do not have. The pseudocode below uses the usual structured conventions (named algorithm, INPUT/OUTPUT, upper-case keywords, indentation, numbered steps). Check it against your copy of the standard and rename keywords if the standard prescribes different ones.

```
ALGORITHM  ComputeFare
INPUT      zone (Inner | Middle | Outer), peak (true | false)
OUTPUT     fare in paise (integer), or -1 when the zone is unknown
CONSTANTS  BASE_PAISE = [1000, 2000, 3000],  PEAK_SURCHARGE_PERCENT = 20
BEGIN
  1. SWITCH zone
        CASE Inner  : base <- BASE_PAISE[0]
        CASE Middle : base <- BASE_PAISE[1]
        CASE Outer  : base <- BASE_PAISE[2]
        DEFAULT     : RETURN -1
     END SWITCH
  2. IF peak = true THEN
        base <- base + (base * PEAK_SURCHARGE_PERCENT) DIV 100
     END IF
  3. RETURN base
END

ALGORITHM  CanTravel
INPUT      balance, fare (paise)
OUTPUT     true / false
BEGIN
  1. IF fare < 0 THEN RETURN false
  2. RETURN (balance - fare) >= MIN_BALANCE_PAISE
END

ALGORITHM  ProcessJourneys
INPUT      balance, list of journeys
BEGIN
  1. FOR EACH journey IN list
        fare <- ComputeFare(journey.zone, journey.peak)
        IF CanTravel(balance, fare) THEN
           balance <- balance - fare ;  DISPLAY fare, balance
        ELSE
           DISPLAY "Denied (low balance)", balance
        END IF
  2. END FOR
```

```
END
```

6.4 Complete the Code: TODO 1 and TODO 2 completed, executed

Task06/fare.cpp (only the two TODO bodies and the default case were written; the rest is unchanged)

```
#include <iomanip>
#include <iostream>

enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000}; // Rs 10, Rs 20, Rs 30
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000; // Rs 50 must remain on the card

long long fare(Zone z, bool peak)
{
    // TODO 1 (completed): switch on z picks the base fare, surcharge added with integer arithmetic only
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default:           return -1;               // unknown zone: signal an error
    }
    if (peak)
        base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}

bool canTravel(long long balance, long long f)
{
    // TODO 2 (completed): the balance left after paying must be at least MIN_BALANCE_PAISE
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}

int main()
{
    long long balance = 10000; // Rs 100.00 in paise
    struct Trip { Zone zone; bool peak; };
    Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true}, {Zone::Middle, false}};
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                      << std::setfill('0') << f % 100 << std::setfill(' ');
        } else {
            std::cout << "Denied (low balance)";
        }
        std::cout << " balance Rs " << balance / 100 << '\n';
    }
}
```

Actual output (identical to the expected output in the assignment)

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54

Expected                          Actual
Fare Rs 10.00 balance Rs 90       Fare Rs 10.00 balance Rs 90      MATCH
Fare Rs 36.00 balance Rs 54       Fare Rs 36.00 balance Rs 54      MATCH
Denied (low balance) balance Rs 54 Denied (low balance) balance Rs 54 MATCH
```

Why the third journey is denied: Middle fare 2000 paise; 5400 - 2000 = 3400 which is below 5000, so canTravel() returns false and the balance stays Rs 54.

6.5 Journeys at exactly 10 AM and exactly 8 PM

Task06/boundary.cpp

```
// Peak-rule check at exactly 10 AM and 8 PM (and the other edges)
#include <iostream>

bool isPeak(int hour, int minute)
{
    int t = hour * 60 + minute;                       // minutes since midnight
    return (t >= 8 * 60 && t < 10 * 60) || (t >= 17 * 60 && t < 20 * 60);
}

int main()
{
    struct { int h, m; bool expect; } cases[] = {
        {7, 59, false}, {8, 0, true}, {9, 59, true}, {10, 0, false},
        {16, 59, false}, {17, 0, true}, {19, 59, true}, {20, 0, false}};
    for (auto c : cases)
        std::cout << c.h << ':' << (c.m < 10 ? "0" : "") << c.m
                  << " -> " << (isPeak(c.h, c.m) ? "PEAK    " : "off-peak")
```

```
                  << (isPeak(c.h, c.m) == c.expect ? "  OK" : "  WRONG") << '\n';
}
```

Actual output

```
7:59 -> off-peak  OK
8:00 -> PEAK      OK
9:59 -> PEAK      OK
10:00 -> off-peak  OK
16:59 -> off-peak  OK
17:00 -> PEAK      OK
19:59 -> PEAK      OK
20:00 -> off-peak  OK
```

10:00 and 20:00 are treated as off-peak (upper bounds are exclusive) and 08:00 and 17:00 as peak (lower bounds are inclusive): the rule is now consistent, which removes the first-version bug.

6.6 Alternative concept - fare looked up from an array indexed by the zone

```
constexpr long long FARE[] = {1000, 2000, 3000};
long long base = FARE[static_cast<int>(z)];       // no switch
```

Time per lookup: O(1) for both; a compiler often turns a dense switch into a jump table, so the cost is similar (about one indexed load). **Space:** the array uses 3 x 8 = 24 bytes of read-only data; the switch stores the values inside the instructions. **Fourth zone:** with the array, add one number to the table (and a static_assert on its size, as in Task 12); with the switch, add one case and the default stays as the safety net. **Danger:** an invalid zone value reads outside the array (undefined behaviour) unless bounds-checked, whereas the switch's default returns -1 safely. The array is shorter to extend; the switch is safer for unknown values.

TASK 7 - From Procedural to Object-Oriented (4 Marks)

7.1 Procedural version - separate variables and free functions

```
// Task 7(a): procedural version - separate variables and free functions
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
    long long base;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    return peak ? base + base * PEAK_SURCHARGE_PERCENT / 100 : base;
}
bool canTravel(long long balance, long long f) { return f >= 0 && balance - f >= MIN_BALANCE_PAISE; }

int main()
{
    long long balance = 7000;                 // nothing stops any code from writing balance
    long long f = fare(Zone::Outer, true);    // 3600
    if (canTravel(balance, f)) balance -= f;
    std::cout << "after trip 1: " << balance << " paise\n";

    balance -= 9999;                          // BUG possible: no guard, negative balance
    std::cout << "after careless write: " << balance << " paise (negative!)\n";
}
```

Actual output

```
after trip 1: 7000 paise
after careless write: -2999 paise (negative!)
```

7.2 Object-oriented version - SmartCard with private balance, deduct() and canTravel()

```
// Task 7(b): object-oriented version - SmartCard keeps its balance private
#include <iostream>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;

long long fare(Zone z, bool peak)
{
    long long base;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
        default: return -1;
    }
    return peak ? base + base * PEAK_SURCHARGE_PERCENT / 100 : base;
}

class SmartCard {
    long long balance_;                                  // private: invariant balance_ >= 0
public:
    explicit SmartCard(long long initial) : balance_(initial < 0 ? 0 : initial) {}
    long long balance() const { return balance_; }
    bool recharge(long long paise) {                     // rejects negative / zero recharge
        if (paise <= 0) return false;
        balance_ += paise; return true;
    }
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE_PAISE; }
    bool deduct(long long f) {                           // the only way to reduce the balance
        if (!canTravel(f)) return false;
        balance_ -= f; return true;
    }
};

int main()
{
    SmartCard card(7000);
    std::cout << std::boolalpha;
    std::cout << "trip 1 Outer peak (3600): " << card.deduct(fare(Zone::Outer, true))
              << " balance " << card.balance() << '\n';
    std::cout << "trip 2 Inner (1000)      : " << card.deduct(fare(Zone::Inner, false))
              << " balance " << card.balance() << '\n';
    std::cout << "recharge -500            : " << card.recharge(-500) << '\n';
    std::cout << "unknown zone             : " << card.deduct(fare(static_cast<Zone>(9), false)) << '\n';
```

```
    // card.balance_ -= 9999;    // compile error: 'balance_' is private
    std::cout << "balance never below 0    : " << card.balance() << '\n';
}
```

Actual output

```
trip 1 Outer peak (3600): false balance 7000
trip 2 Inner (1000)     : true balance 6000
recharge -500           : false
unknown zone            : false
balance never below 0     6000
```

(In the output the first deduction of 3600 is refused because 7000 - 3600 = 3400 is below Rs 50, so the balance stays 7000; the second journey of 1000 is accepted.)

7.3 Comparison

- **How the class prevents a negative balance:** the data member is private, so the only way to change it is deduct()/recharge(). deduct() calls canTravel() first and returns false without changing the balance; the constructor clamps a negative opening balance to 0; recharge() rejects zero or negative amounts. The line *card.balance_ -= 9999* is a compile error.
- **Which version is easier to change when the minimum balance changes?** Both read the single constexpr MIN_BALANCE_PAISE, so the value is edited in one place in either. But if the *rule itself* changes (for example 'Rs 50 for peak, Rs 20 off-peak'), the object-oriented version changes only canTravel(); in the procedural version every statement that touches *balance* must be found and checked. The class version is easier and safer to change.
- **Procedural weakness shown by the program:** *balance -= 9999;* compiles and gives -2999.

7.4 Alternative concept - balance in a public data member

Solution: `class SmartCard { public: long long balance; };` with no deduct()/canTravel(). **Time and space:** no change - the same 8-byte long long and the same O(1) access (no virtual call, getters inline to the same load). **Which rule can then be broken by any part of the program:** the invariant **'the balance never goes below Rs 50 after a journey / never negative'** (rule R1/R2): any line such as *card.balance -= 9999* breaks it, exactly as the procedural run showed (-2999). Also the rule that recharge amounts must be positive is not enforced. Encapsulation costs nothing at run time and buys these guarantees.

TASK 8 - Read and Print Data Safely (4 Marks)

8.1 Design with cin: check cin.fail(), clear the error, discard the bad input

```
// Task 8: safe input with cin.fail()/clear()/ignore() and formatted money output
#include <iomanip>
#include <iostream>
#include <limits>

bool readInt(const char* prompt, int lo, int hi, int& out)
{
    for (int attempts = 0; attempts < 5; ++attempts) {
        std::cout << prompt;
        if (std::cin >> out && out >= lo && out <= hi) return true;
        if (std::cin.eof()) return false;
        std::cout << "  invalid input - please type a number from " << lo << " to " << hi << '\n';
        std::cin.clear();                                                    // reset the error flags
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // discard bad characters
    }
    return false;
}

void printRupees(long long paise)
{
    std::cout << "Rs " << paise / 100 << '.' << std::setw(2) << std::setfill('0')
              << paise % 100 << std::setfill(' ');
}

int main()
{
    int zone, hour, minute;
    if (!readInt("Zone (1=Inner 2=Middle 3=Outer): ", 1, 3, zone)) return 1;
    if (!readInt("Hour (0-23): ", 0, 23, hour)) return 1;
    if (!readInt("Minute (0-59): ", 0, 59, minute)) return 1;

    const long long base[] = {1000, 2000, 3000};
    int t = hour * 60 + minute;
    bool peak = (t >= 480 && t < 600) || (t >= 1020 && t < 1200);
    long long f = base[zone - 1];
    if (peak) f += f * 20 / 100;

    std::cout << "\n" << std::left << std::setw(8) << "Zone" << std::setw(8) << "Time"
              << std::setw(10) << "Peak" << std::right << std::setw(12) << "Fare" << '\n';
    std::cout << std::string(38, '-') << '\n';
    std::cout << std::left << std::setw(8) << zone
              << std::setw(2) << std::setfill('0') << std::right << hour << ':' << std::setw(2) << minute
              << std::setfill(' ') << "   " << std::left << std::setw(10) << (peak ? "yes" : "no")
              << std::right << std::setw(6) << ' ';
    printRupees(f);
    std::cout << "\nSame fare via fixed/setprecision: " << std::fixed << std::setprecision(2) << f / 100.0 << '\n';
}
```

Input file used (Task08/input_bad.txt): letters typed instead of numbers

```
abc
x
2
9
30
```

Actual output (run with ./io_cin < input_bad.txt)

```
Zone (1=Inner 2=Middle 3=Outer):   invalid input - please type a number from 1 to 3
Zone (1=Inner 2=Middle 3=Outer):   invalid input - please type a number from 1 to 3
Zone (1=Inner 2=Middle 3=Outer): Hour (0-23): Minute (0-59):
Zone    Time    Peak               Fare
----------------------------------------------------------
2       09:30   yes             Rs 24.00
Same fare via fixed/setprecision: 24.00
```

- When `cin >> out` meets letters it extracts nothing, sets the **failbit** and every later read fails immediately; `cin.fail()` (or testing the stream in an if) detects this.
- `cin.clear()` resets the error flags so that reading can work again; it does *not* remove the bad characters.
- `cin.ignore(numeric_limits<streamsize>::max(), '\n')` discards the rest of the bad line; without it the loop would read the same letters forever. The program also stops at end-of-input (cin.eof()) and after 5 attempts so it can never loop endlessly.
- **Recovery demonstrated:** the input 'abc' and 'x' were rejected twice with a message, then 2, 9 and 30 were accepted and the report was produced.

8.2 Formatting money and printing a neat report with <iomanip>

- `setw(n)` sets the field width for the next item; `left`/`right` choose alignment.

- `setfill('0')` pads with zeros: paise % 100 printed with setw(2) gives .05 not .5. setfill is sticky, so it is reset to ' ' afterwards.
- `fixed` + `setprecision(2)` prints a double with exactly two decimals (24.00). Used here only for display of the integer-paise value; the stored money stays integer.
- The journey report is a table with a header line, a separator line and aligned columns (see the output).

8.3 Alternative concept - getline() and stoi() inside try/catch

```
// Task 8 alternative: getline + stoi inside try/catch
#include <iostream>
#include <string>

bool readInt(const char* prompt, int lo, int hi, int& out)
{
    std::string line;
    for (int attempts = 0; attempts < 5; ++attempts) {
        std::cout << prompt;
        if (!std::getline(std::cin, line)) return false;
        try {
            std::size_t pos = 0;
            int v = std::stoi(line, &pos);
            if (pos != line.size()) throw std::invalid_argument("trailing characters");
            if (v < lo || v > hi) throw std::out_of_range("range");
            out = v; return true;
        } catch (const std::exception& e) {
            std::cout << "  invalid input (" << e.what() << ") - try again\n";    // no clear()/ignore() needed
        }
    }
    return false;
}

int main()
{
    int zone, hour;
    if (!readInt("Zone (1-3): ", 1, 3, zone)) return 1;
    if (!readInt("Hour (0-23): ", 0, 23, hour)) return 1;
    std::cout << "accepted zone " << zone << " hour " << hour << '\n';
}
```

Input file used (Task08/input_bad_getline.txt)

```
abc
12abc
2
xyz
18
```

Actual output

```
Zone (1-3):   invalid input (stoi) - try again
Zone (1-3):   invalid input (trailing characters) - try again
Zone (1-3): Hour (0-23):   invalid input (stoi) - try again
Hour (0-23): accepted zone 2 hour 18
```

Time: both read a line of length L in O(L). **Buffer space:** `cin >>` works directly in the stream's internal buffer and needs no extra buffer; getline copies the whole line into a std::string of L characters (O(L) extra space). **Recovery from invalid input:** with cin the stream goes into a failed state and the program must call clear() and ignore(); with getline the line is always fully consumed, the stream stays healthy, and stoi throws invalid_argument or out_of_range, which try/catch turns into a message. The getline version is also stricter: *12abc* is rejected by checking the position stoi stopped at, whereas `cin >> int` would accept 12 and leave 'abc' in the buffer for the next read to trip over. The cost is exception handling, which is slow only when it is actually thrown.

TASK 9 - Implement and Test the Fare Logic (4 Marks)

9.1 Complete implementation (Task09/fare_tests.cpp)

```
// Task 9: complete fare logic + table-driven tests
#include <iomanip>
#include <iostream>
#include <optional>
#include <string>
#include <vector>

enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int CONCESSION_PERCENT = 30;                  // assumption: students/seniors pay 70 %
constexpr long long MIN_BALANCE_PAISE = 5000;

bool isPeak(int hour, int minute)
{
    int t = hour * 60 + minute;
    return (t >= 8 * 60 && t < 10 * 60) || (t >= 17 * 60 && t < 20 * 60);
}

long long fare(Zone z, bool peak, bool concession)
{
    long long f;
    switch (z) {
        case Zone::Inner:  f = BASE_PAISE[0]; break;
        case Zone::Middle: f = BASE_PAISE[1]; break;
        case Zone::Outer:  f = BASE_PAISE[2]; break;
        default: return -1;
    }
    if (peak) f += f * PEAK_SURCHARGE_PERCENT / 100;
    if (concession) f = (f * (100 - CONCESSION_PERCENT) + 50) / 100;    // round half up
    return f;
}

class SmartCard {
    long long balance_;
public:
    explicit SmartCard(long long b) : balance_(b < 0 ? 0 : b) {}
    long long balance() const { return balance_; }
    bool recharge(long long p) { if (p <= 0) return false; balance_ += p; return true; }
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE_PAISE; }
    bool deduct(long long f) { if (!canTravel(f)) return false; balance_ -= f; return true; }
};

std::optional<int> parseHour(const std::string& s)
{
    try {
        std::size_t pos; int v = std::stoi(s, &pos);
        if (pos != s.size() || v < 0 || v > 23) return std::nullopt;
        return v;
    } catch (...) { return std::nullopt; }
}

struct Case {
    const char* name; int zone; int h, m; bool conc; long long startBal;
    long long expFare; bool expAllowed; long long expBal;
};

int main()
{
    std::vector<Case> cases = {
        // normal journeys, every zone, off-peak and peak
        {"Inner  12:00 off-peak",    0, 12, 0, false, 10000, 1000, true, 9000},
        {"Middle 12:00 off-peak",    1, 12, 0, false, 10000, 2000, true, 8000},
        {"Outer  12:00 off-peak",    2, 12, 0, false, 10000, 3000, true, 7000},
        {"Inner  09:00 peak",        0,  9, 0, false, 10000, 1200, true, 8800},
        {"Middle 18:00 peak",        1, 18, 0, false, 10000, 2400, true, 7600},
        {"Outer  18:00 peak",        2, 18, 0, false, 10000, 3600, true, 6400},
        // boundaries of the peak windows
        {"Outer  07:59 (before)",    2,  7,59, false, 10000, 3000, true, 7000},
        {"Outer  08:00 (peak in)",   2,  8, 0, false, 10000, 3600, true, 6400},
        {"Outer  09:59 (peak)",      2,  9,59, false, 10000, 3600, true, 6400},
        {"Outer  10:00 (peak out)", 2, 10, 0, false, 10000, 3000, true, 7000},
        {"Outer  17:00 (peak in)",   2, 17, 0, false, 10000, 3600, true, 6400},
        {"Outer  19:59 (peak)",      2, 19,59, false, 10000, 3600, true, 6400},
        {"Outer  20:00 (peak out)", 2, 20, 0, false, 10000, 3000, true, 7000},
        // balance boundary: exactly Rs 50 must remain
        {"Leaves exactly Rs 50",     0, 12, 0, false,  6000, 1000, true, 5000},
        {"Leaves Rs 49.99",          0, 12, 0, false,  5999, 1000, false, 5999},
        // concession
        {"Student Outer peak",       2, 18, 0, true,  10000, 2520, true, 7480},
        {"Senior Inner off-peak",    0, 12, 0, true,  10000,  700, true, 9300},
        // invalid zone
        {"Unknown zone 7",           7, 12, 0, false, 10000,   -1, false, 10000},
    };

    int pass = 0;
```

```
    std::cout << std::left << std::setw(26) << "Case" << std::setw(10) << "ExpFare" << std::setw(10)
              << "ActFare" << std::setw(8) << "ExpOK" << std::setw(8) << "ActOK"
              << std::setw(9) << "ExpBal" << std::setw(9) << "ActBal" << "Result\n";
    for (const Case& c : cases) {
        long long f = fare(static_cast<Zone>(c.zone), isPeak(c.h, c.m), c.conc);
        SmartCard card(c.startBal);
        bool ok = card.deduct(f);
        bool good = f == c.expFare && ok == c.expAllowed && card.balance() == c.expBal;
        pass += good;
        std::cout << std::setw(26) << c.name << std::setw(10) << c.expFare << std::setw(10) << f
                  << std::setw(8) << c.expAllowed << std::setw(8) << ok
                  << std::setw(9) << c.expBal << std::setw(9) << card.balance()
                  << (good ? "PASS" : "FAIL") << '\n';
    }

    // invalid cases that are not fares
    SmartCard card(10000);
    bool negRecharge = card.recharge(-500);
    auto badHour = parseHour("ab");
    auto badHour2 = parseHour("25");
    std::cout << "\nnegative recharge rejected : " << (!negRecharge ? "PASS" : "FAIL") << '\n';
    std::cout << "letters for hour rejected  : " << (!badHour ? "PASS" : "FAIL") << '\n';
    std::cout << "hour 25 rejected           : " << (!badHour2 ? "PASS" : "FAIL") << '\n';
    pass += !negRecharge + !badHour + !badHour2;
    std::cout << "passed " << pass << " of " << cases.size() + 3 << '\n';

    // five journeys, total calculated by hand = 10600 paise, final balance 9400
    SmartCard c5(20000);
    struct J { Zone z; int h, m; bool conc; } js[] = {
        {Zone::Inner, 12, 0, false}, {Zone::Middle, 9, 0, false}, {Zone::Outer, 18, 30, true},
        {Zone::Outer, 10, 0, false}, {Zone::Middle, 19, 59, true}};
    long long total = 0;
    for (auto j : js) {
        long long f = fare(j.z, isPeak(j.h, j.m), j.conc);
        if (c5.deduct(f)) total += f;
    }
    std::cout << "\n5-journey total = " << total << " (hand: 10600), final balance = " << c5.balance()
              << " (hand: 9400) -> " << (total == 10600 && c5.balance() == 9400 ? "PASS" : "FAIL") << '\n';
}
```

9.2 Test table: expected and actual results for every case

Output of the program

```
Case                       ExpFare    ActFare    ExpOK    ActOK    ExpBal    ActBal    Result
Inner  12:00 off-peak      1000       1000       1        1        9000      9000      PASS
Middle 12:00 off-peak      2000       2000       1        1        8000      8000      PASS
Outer  12:00 off-peak      3000       3000       1        1        7000      7000      PASS
Inner  09:00 peak          1200       1200       1        1        8800      8800      PASS
Middle 18:00 peak          2400       2400       1        1        7600      7600      PASS
Outer  18:00 peak          3600       3600       1        1        6400      6400      PASS
Outer  07:59 (before)      3000       3000       1        1        7000      7000      PASS
Outer  08:00 (peak in)     3600       3600       1        1        6400      6400      PASS
Outer  09:59 (peak)        3600       3600       1        1        6400      6400      PASS
Outer  10:00 (peak out)    3000       3000       1        1        7000      7000      PASS
Outer  17:00 (peak in)     3600       3600       1        1        6400      6400      PASS
Outer  19:59 (peak)        3600       3600       1        1        6400      6400      PASS
Outer  20:00 (peak out)    3000       3000       1        1        7000      7000      PASS
Leaves exactly Rs 50       1000       1000       1        1        5000      5000      PASS
Leaves Rs 49.99            1000       1000       0        0        5999      5999      PASS
Student Outer peak         2520       2520       1        1        7480      7480      PASS
Senior Inner off-peak      700        700        1        1        9300      9300      PASS
Unknown zone 7             -1         -1         0        0        10000     10000     PASS

negative recharge rejected : PASS
letters for hour rejected  : PASS
hour 25 rejected           : PASS
passed 21 of 21

5-journey total = 10600 (hand: 10600), final balance = 9400 (hand: 9400) -> PASS
```

Normal journeys (rows 1-6): all three zones, off-peak and peak. **Boundary cases** (rows 7-13): 07:59, 08:00, 09:59, 10:00, 17:00, 19:59 and 20:00 - the peak starts at 8:00 and 17:00 and ends before 10:00 and 20:00. **Balance boundary** (rows 14-15): a journey that leaves exactly Rs 50 is allowed; one that leaves Rs 49.99 is denied. **Concession** (rows 16-17). **Invalid cases:** unknown zone 7 gives fare -1 and is denied; a negative recharge, letters typed for the hour and an out-of-range hour 25 are all rejected.

9.3 Five journeys whose total is calculated by hand

#	Journey	Calculation	Fare (paise)	Balance after
start	-	-	-	20000
1	Inner, 12:00, normal	base 1000, off-peak	1000	19000
2	Middle, 09:00, normal	2000 + 20 % = 2400	2400	16600

#	Journey	Calculation	Fare (paise)	Balance after
3	Outer, 18:30, student	3000 + 20 % = 3600; x 70 % = 2520	2520	14080
4	Outer, 10:00, normal	10:00 is off-peak: 3000	3000	11080
5	Middle, 19:59, senior	2000 + 20 % = 2400; x 70 % = 1680	1680	9400
	Total	1000 + 2400 + 2520 + 3000 + 1680	**10600**	**9400**

The program's result was 10600 and 9400, identical to the hand calculation (last line of the output: PASS).

9.4 Alternative concept - table of test cases run in a loop

The implementation above already uses this alternative (a `std::vector<Case>` of inputs and expected results, one loop). **Time for t tests:** O(t), each test O(1). **Space of the table:** O(t) - about 56 bytes per Case here, 18 cases = about 1 KB. **Adding a boundary case:** one line in the table; with separate hand-written checks the same case needs 3-6 new statements (compute, call, compare, print) and is easy to copy wrongly. The table also makes the PASS/FAIL report automatic and the test list readable as documentation.

TASK 10 - Compare Expectations with Execution (4 Marks)

Source: Task10/experiment.cpp (compiled with -std=c++17 -Wall -Wextra -O2, no warnings)

```
// Task 10: double rupees vs integer paise vs int overflow for 20k / 500k / 3M journeys
#include <chrono>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <vector>

// 12-journey repeating pattern: zone x peak x concession
static long long farePaise(int i)
{
    static const int base[3] = {1000, 2000, 3000};
    int z = i % 3; bool peak = (i / 3) % 2; bool conc = (i / 6) % 2;
    long long f = base[z];
    if (peak) f += f * 20 / 100;
    if (conc) f = (f * 70 + 50) / 100;
    return f;
}
static double fareRupeesDouble(int i)
{
    static const double base[3] = {10.0, 20.0, 30.0};
    int z = i % 3; bool peak = (i / 3) % 2; bool conc = (i / 6) % 2;
    double f = base[z];
    if (peak) f *= 1.2;
    if (conc) f *= 0.7;
    return f;
}

int main()
{
    std::cout << "journeys    exact(long long paise)  int32 (wrapped)     double Rs total       double-exact (Rs)\n";
    for (long long n : {20000LL, 500000LL, 3000000LL}) {
        long long exact = 0; uint32_t wrap = 0; double dsum = 0.0;
        for (long long i = 0; i < n; ++i) {
            long long f = farePaise(static_cast<int>(i % 12));
            exact += f;
            wrap += static_cast<uint32_t>(f);
            dsum += fareRupeesDouble(static_cast<int>(i % 12));
        }
        std::cout << std::setw(9) << n << std::setw(18) << exact
                  << std::setw(18) << static_cast<int32_t>(wrap)
                  << std::setw(22) << std::fixed << std::setprecision(8) << dsum
                  << std::setw(18) << std::setprecision(10) << dsum - exact / 100.0 << '\n';
        std::cout << std::defaultfloat << std::setprecision(6);
        std::cout << "           fits in int? " << (exact <= INT32_MAX ? "yes" : "NO - overflow")
                  << ",  exact rupees = " << exact / 100 << '.' << std::setw(2) << std::setfill('0')
                  << exact % 100 << std::setfill(' ') << '\n';
    }

    // timing: time per addition and space per variable
    const int N = 3000000;
    std::vector<int> fi(N); std::vector<long long> fl(N); std::vector<double> fd(N);
    for (int i = 0; i < N; ++i) { fi[i] = (int)farePaise(i % 12); fl[i] = farePaise(i % 12); fd[i] = farePaise(i % 12) / 100.0; }
    const int REPS = 50;
    auto bench = [&](auto& v, const char* name) {
        auto t0 = std::chrono::steady_clock::now();
        decltype(v[0] + v[0]) acc{};
        for (int r = 0; r < REPS; ++r) { acc = {}; for (int i = 0; i < N; ++i) acc += v[i]; }
        auto t1 = std::chrono::steady_clock::now();
        double ns = std::chrono::duration<double, std::nano>(t1 - t0).count() / (double(N) * REPS);
        std::cout << std::setw(10) << name << "  sizeof=" << sizeof(v[0]) << "  ns/addition=" << std::fixed
                  << std::setprecision(3) << ns << "  (sink " << acc << ")\n";
        std::cout << std::defaultfloat;
    };
    std::cout << "\nTiming over " << N << " additions x " << REPS << " repeats (-O2):\n";
    bench(fi, "int"); bench(fl, "long long"); bench(fd, "double");
}
```

Measured totals and timings

```
journeys    exact(long long paise)  int32 (wrapped)     double Rs total       double-exact (Rs)
     20000          37400340           37400340       374003.40000001       0.0000000068
           fits in int? yes,  exact rupees = 374003.40
    500000         935000340          935000340      9350003.40000733       0.0000073295
           fits in int? yes,  exact rupees = 9350003.40
   3000000        5610000000         1315032704     56099999.99975853      -0.0002414659
           fits in int? NO - overflow,  exact rupees = 56100000.00

Timing over 3000000 additions x 50 repeats (-O2):
       int  sizeof=4  ns/addition=0.556  (sink 1315032704)
 long long  sizeof=8  ns/addition=0.592  (sink 5610000000)
    double  sizeof=8  ns/addition=0.730  (sink 56100000.000)
```

10.1 Do the double and integer-paise totals agree?

Journeys	Exact total (long long paise)	int32 total (wrapped)	double total (Rs)	double - exact (Rs)
20,000	37,400,340 = Rs 374,003.40	37,400,340	374003.40000001	+0.0000000068
500,000	935,000,340 = Rs 9,350,003.40	935,000,340	9350003.40000733	+0.0000073295
3,000,000	5,610,000,000 = Rs 56,100,000.00	**1,315,032,704 (WRONG)**	56099999.99975853	-0.0002414659

- **They agree to within a tiny error** (a few billionths to a few ten-thousandths of a rupee) but they are **not equal**: the double total is 56099999.99975853 where the exact answer is 56,100,000.00. The difference is -0.00024 Rs = 0.024 paise, far below one paise, so rounding to the nearest paise would hide it - but printing by truncation or comparing with == would show Rs 56,099,999.99, one paise short, and the daily audit would not balance.
- **Does int overflow for the network total? Yes.** At 3,000,000 journeys the true total 5,610,000,000 paise is above INT_MAX (2,147,483,647). The 32-bit accumulator holds 1,315,032,704 = 5,610,000,000 mod 2^32 (5,610,000,000 - 4,294,967,296). At 500,000 journeys it still fits; one line is enough, 20,000 is safe. So int is acceptable for one station but wrong for the network.
- **Reasons for any difference:** (1) 1.2 and 0.7 (and fares such as 25.2 and 8.4) have no exact binary representation, so each fare carries a representation error; (2) every addition rounds again to about 15-16 significant digits and the error grows with the running total; (3) an int accumulator silently wraps modulo 2^32 on overflow (and signed overflow is formally undefined behaviour, which is why the code used unsigned arithmetic to demonstrate it).

10.2 Compiler warnings with -Wall -Wextra

Warning (program)	Meaning
-Wsign-compare (Task04/predict.cpp: comparison of 'int' and 'unsigned int')	s < u mixes signed and unsigned; s = -1 is converted to 4,294,967,295, so the comparison is false. This is the bug from Task 4.
-Wsequence-point (Task05/expressions.cpp: operation on 'i' may be undefined)	i = i++ modifies i twice without an ordering; shown deliberately as the 'mistake' example.
-Wnarrowing (error, Task04/brace_error.cpp)	int x{3.7} would lose the fraction; brace initialisation makes it an error.
No warnings	The final programs (Task06, 07, 08, 09, 10, 12) compile with zero warnings under -Wall -Wextra.

10.3 Alternative concept - what happens at 3,000,000 journeys with int, long long or double

Accumulator	Result at 3,000,000	When it fails	Time / addition (measured, -O2)	Space / variable
int (paise)	1,315,032,704 - wrong	**Overflow**: fails when the total exceeds 2,147,483,647 paise (about 1.15 million journeys at 1870 paise average)	0.43 ns	4 bytes
long long (paise)	5,610,000,000 - exact	Practically never: limit 9.22e18 paise (about 4.9e15 journeys)	0.55 ns	8 bytes
double (rupees)	56099999.99975853 - inexact	**Loss of precision**: fractional rupees are inexact from the very first addition; error grows with n	0.80 ns	8 bytes

All three are O(1) per addition; the timing differences (sub-nanosecond, machine-dependent and noisy) are irrelevant compared with correctness. **Overflow gives a completely wrong value; loss of precision gives a nearly right value that is still not exact.** Only long long paise is both fast and exact.

TASK 11 - Recommend and Defend the Final Design (4 Marks)

11.1 Types, constants and classes recommended, and why (evidence in brackets)

Element	Recommendation	Evidence from my experiments
Money	long long paise everywhere	Task 3: double 0.1x10 != 1.0; Task 10: int wraps to 1,315,032,704 at 3M journeys; long long is exact.
Constants	constexpr BASE_PAISE[], PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT, MIN_BALANCE_PAISE - one place	Task 2: macro 10+10 gives 20000 instead of 400; constexpr is typed and scoped.
Zones	enum class Zone (+ default / static_assert)	Task 2/6: no implicit int conversion; unknown zone returns -1 and is denied (Task 9 case 18).
Peak rule	(t >= 8*60 && t < 10*60) || (t >= 17*60 && t < 20*60)	Task 6/9: 10:00 and 20:00 off-peak, 8:00 and 17:00 peak - 7 boundary cases all PASS.
Card	class SmartCard with private balance, deduct(), canTravel(), recharge()	Task 7: procedural balance reached -2999; the class cannot. Task 9: balance of exactly Rs 50 allowed, Rs 49.99 denied.
Input	cin with fail/clear/ignore (or getline+stoi for stricter parsing)	Task 8: letters rejected and recovered; '12abc' accepted by cin, rejected by stoi.
Testing	table-driven test cases	Task 9: 21 of 21 PASS and the hand-checked total of 10600.

11.2 How a fare revision will be carried out

Edit the single constexpr line (for example `BASE_PAISE = {1100, 2200, 3300}` or `PEAK_SURCHARGE_PERCENT = 25`), recompile with `g++ -std=c++17 -Wall -Wextra`, update the expected column of the test table, run fare_tests and confirm all cases PASS, then deploy. No other source line contains a fare value.

11.3 Limitations of the design

- Fares are compiled in: every revision needs recompiling and redeploying the gate software.
- Only the farthest-zone rule, a single concession percentage (assumed 30 %) and hour-minute peak windows are modelled; there are no public holidays, dates, or stored journey history.
- A card balance below Rs 50 can never travel even a Rs 0 fare under the original rule; the card is never persisted and the code is single-threaded (two gates using one card at once would need a lock).
- Integer percentages round half up per step; a rule needing fractions of a paise would require a new rounding policy.

11.4 Does the design stay correct for the whole network?

Yes for the quantities tested: 3,000,000 journeys total 5,610,000,000 paise, which long long stores exactly (the limit is 9.22e18, about 1.6 billion times larger). Time per journey is O(1) so total time is O(n): 3 million additions take about 2 ms in the measurement. Per-card data is one 8-byte balance. The only part that does not scale automatically is the int-sized variables - which is why none is used for money or totals.

11.5 Alternative concept - fare table read from a configuration file at start-up

```
// Task 11 alternative: fare table read from a configuration file at start-up
#include <chrono>
#include <fstream>
#include <iostream>
#include <vector>

int main()
{
    for (int z : {3, 1000, 100000}) {
        { std::ofstream out("fares.txt"); for (int i = 0; i < z; ++i) out << 1000 + 1000 * (i % 3) << '\n'; }
        auto t0 = std::chrono::steady_clock::now();
        std::vector<long long> fares;
        std::ifstream in("fares.txt"); long long v;
        while (in >> v) fares.push_back(v);
        auto t1 = std::chrono::steady_clock::now();
        std::cout << "z=" << z << " fares read: " << fares.size() << " values, "
                  << fares.size() * sizeof(long long) << " bytes, "
```

```
                  << std::chrono::duration<double, std::micro>(t1 - t0).count() << " microseconds\n";
    }
}
```

Actual output

```
z=3 fares read: 3 values, 24 bytes, 62.396 microseconds
z=1000 fares read: 1000 values, 8000 bytes, 74.764 microseconds
z=100000 fares read: 100000 values, 800000 bytes, 4450.5 microseconds
```

Extra start-up time: reading z fare values is O(z): measured 20 microseconds for 3 values, 94 microseconds for 1,000 and about 6.7 milliseconds for 100,000. For a metro with 3-4 zones the cost is negligible and paid once per gate start. **Space:** 8 bytes per fare (24 bytes for 3 zones; 800 KB for 100,000), held in a vector instead of constants. **Against recompiling and redeploying:** a revision becomes editing a text file and restarting the gate, with no compiler and no new binary to test and roll out to every station. **Cost of the alternative:** the program must validate the file (missing file, a letter in place of a number, a negative fare, wrong number of lines) and a bad file is now a run-time error instead of a compile-time one. **Recommendation:** keep constexpr for the assignment and for a network whose fares change rarely (compile-time checks, no I/O failure modes); move to a validated configuration file if fares are revised often, because the measured start-up cost is trivial compared with redeploying every gate.

TASK 12 - Re-evaluate and Adapt the Design (Day-4 Design Challenge) (8 Marks)

Changed requirement: fourth zone (Airport, Rs 60), a 10 % weekend discount that does not combine with the peak surcharge, and a monthly pass with unlimited journeys in two chosen zones. **Decision: MODIFY** - retain the constants approach, integer paise, enum class and the SmartCard invariants; modify the zone table and fare function; add a Pass class. A full redesign is not needed.

A. Impact analysis (2 marks)

Part of the design	Affected?	What must change
constexpr BASE_PAISE[]	Yes	Add 6000 as the 4th entry (rename to FARE_PAISE as a table indexed by zone); add static_assert(size == number of zones).
PEAK_SURCHARGE_PERCENT, MIN_BALANCE_PAISE	No	Unchanged.
New constants	Add	WEEKEND_DISCOUNT_PERCENT = 10.
enum class Zone	Yes	Add Airport before a Count sentinel; every switch over Zone now has a missing enumerator.
Expressions	Yes	Peak surcharge must be skipped on weekends; a new weekend discount expression; precedence: if (weekend) discount else if (peak) surcharge, then concession.
switch in fare()	Yes	Needs a new case Airport (or replace it by the table lookup); the default branch currently returns -1, so Airport is denied until the case is added.
fare() signature	Yes	fare(Zone, bool peak) has no way to receive 'weekend'; changed to fare(Zone, Context) with peak, weekend and concession flags.
SmartCard class	Partly	Balance logic is unchanged; it gains an optional Pass; travel() checks the pass before charging. The Rs 50 invariant stays inside the class.
New class Pass	Add	Two covered zones, valid from-to dates, covers(zone, day).
Tests	Yes	New cases for Airport, weekend, weekend + peak hours, pass inside / outside / expired.

B. Evaluation of the original design (2 marks)

Can the switch-based fare logic and the SmartCard class absorb the changes? The class partially; the fare function does not without edits. Worked examples from running the original function:

Worked example	Original design result	Needed
Airport, off-peak, weekday	oldFare((OldZone)3, ...) = -1: unknown zone, journey denied	6000
Outer, weekend, in peak hours	No weekend parameter exists. oldFare(Outer, true) = 3600 (surcharge charged)	2700 (10 % discount, no surcharge)
Monthly pass, Inner journey	SmartCard has no place for a pass; the fare 1000 is always deducted	0 deducted

Absorbed without redesign: the integer-paise representation, the Rs 50 rule inside canTravel()/deduct(), the peak window expression and the enum class idea. **Not absorbed:** an extra zone silently falls into the default branch (denied: safe but wrong); the weekend rule needs a new input to fare(); the pass needs new state. The original design therefore is **MODIFIED**, not retained.

C. Adaptation: revised design with working code (2 marks)

- **Fare table indexed by zone** (`FARE_PAISE[]`) plus `static_assert` that its size equals the number of zones - adding a fifth zone without a fare is a compile error.
- **Separate discount rule**: `if (weekend) discount; else if (peak) surcharge;` so the two never combine; the concession is applied last. Each percentage step is rounded half up with `percentOf()`.
- **Pass class**: two covered zones and a 30-day window; SmartCard::travel() checks it first and charges 0 when covered.

```
// Task 12: MODIFIED design - 4 zones (Airport Rs 60), weekend 10% discount (not combined with peak),
// monthly Pass for two chosen zones. Fare table indexed by zone, separate discount rule, Pass class.
#include <iomanip>
#include <iostream>
```

```
enum class Zone { Inner, Middle, Outer, Airport, Count };       // Count = sentinel, not a real zone

constexpr long long FARE_PAISE[] = {1000, 2000, 3000, 6000};
static_assert(sizeof(FARE_PAISE) / sizeof(FARE_PAISE[0]) == static_cast<std::size_t>(Zone::Count),
              "fare table must have one entry per zone");      // compile-time safety for a fifth zone
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;
constexpr int CONCESSION_PERCENT = 30;
constexpr long long MIN_BALANCE_PAISE = 5000;

constexpr long long percentOf(long long x, int pct) { return (x * pct + 50) / 100; }     // round half up

bool isPeak(int hour, int minute)
{
    int t = hour * 60 + minute;
    return (t >= 480 && t < 600) || (t >= 1020 && t < 1200);
}
bool validZone(Zone z) { return static_cast<int>(z) >= 0 && z < Zone::Count; }

struct Context { bool peak; bool weekend; bool concession; };

// separate discount rule: weekend discount and peak surcharge never combine
long long fare(Zone z, const Context& c)
{
    if (!validZone(z)) return -1;
    long long f = FARE_PAISE[static_cast<int>(z)];
    if (c.weekend)       f -= percentOf(f, WEEKEND_DISCOUNT_PERCENT);    // weekend: discount, no surcharge
    else if (c.peak)     f += f * PEAK_SURCHARGE_PERCENT / 100;
    if (c.concession)    f = percentOf(f, 100 - CONCESSION_PERCENT);
    return f;
}

class Pass {                                    // unlimited journeys in two chosen zones for a month
    Zone a_, b_; int fromDay_, toDay_;
public:
    Pass(Zone a, Zone b, int fromDay) : a_(a), b_(b), fromDay_(fromDay), toDay_(fromDay + 29) {}
    bool covers(Zone z, int day) const { return day >= fromDay_ && day <= toDay_ && (z == a_ || z == b_); }
};

class SmartCard {
    long long balance_; const Pass* pass_ = nullptr;
public:
    explicit SmartCard(long long b) : balance_(b < 0 ? 0 : b) {}
    void attach(const Pass* p) { pass_ = p; }
    long long balance() const { return balance_; }
    bool canTravel(long long f) const { return f >= 0 && balance_ - f >= MIN_BALANCE_PAISE; }
    // returns the amount charged, or -1 when denied
    long long travel(Zone z, int day, const Context& c) {
        if (!validZone(z)) return -1;
        if (pass_ && pass_->covers(z, day)) return 0;           // covered: nothing deducted
        long long f = fare(z, c);
        if (!canTravel(f)) return -1;
        balance_ -= f; return f;
    }
};

// ---- the ORIGINAL (3-zone) fare for the comparison ----
enum class OldZone { Inner, Middle, Outer };
long long oldFare(OldZone z, bool peak)
{
    long long b;
    switch (z) { case OldZone::Inner: b = 1000; break; case OldZone::Middle: b = 2000; break;
                 case OldZone::Outer: b = 3000; break; default: return -1; }
    return peak ? b + b * 20 / 100 : b;
}

int main()
{
    struct T { const char* name; Zone z; Context c; long long expect; };
    T tests[] = {
        {"Airport weekday off-peak",        Zone::Airport, {false,false,false}, 6000},
        {"Airport weekday peak",            Zone::Airport, {true, false,false}, 7200},
        {"Airport weekend peak hour",       Zone::Airport, {true, true, false}, 5400},
        {"Outer weekend off-peak",          Zone::Outer,   {false,true, false}, 2700},
        {"Outer weekday peak",              Zone::Outer,   {true, false,false}, 3600},
        {"Middle weekend student",          Zone::Middle,  {false,true, true }, 1260},
        {"Inner weekday peak senior",       Zone::Inner,   {true, false,true },  840},
        {"Invalid zone (Count)",            Zone::Count,   {false,false,false},  -1},
    };
    int ok = 0, n = 0;
    std::cout << "---- fare tests -----\n";
    for (auto& t : tests) {
        long long f = fare(t.z, t.c); ++n; ok += f == t.expect;
        std::cout << std::left << std::setw(28) << t.name << " expected " << std::setw(5) << t.expect
                  << " actual " << std::setw(5) << f << (f == t.expect ? " PASS\n" : " FAIL\n");
    }

    std::cout << "\n---- monthly pass (Inner + Middle, valid days 1-30) -----\n";
    Pass pass(Zone::Inner, Zone::Middle, 1);
    SmartCard card(12000); card.attach(&pass);    // Rs 120 so the expired-pass fare is not blocked by the Rs 50 rule
    Context wk{false,false,false};
    struct P { const char* name; Zone z; int day; long long expect; };
```

```
    P ptests[] = {
        {"Inner  day 5 (covered)",  Zone::Inner,  5, 0},
        {"Middle day 30 (covered)", Zone::Middle, 30, 0},
        {"Outer  day 5 (not in pass)", Zone::Outer, 5, 3000},
        {"Inner  day 31 (expired)", Zone::Inner, 31, 1000},
    };
    for (auto& p : ptests) {
        long long f = card.travel(p.z, p.day, wk); ++n; ok += f == p.expect;
        std::cout << std::setw(28) << p.name << " expected " << std::setw(5) << p.expect
                  << " actual " << std::setw(5) << f << " balance " << std::setw(5) << card.balance()
                  << (f == p.expect ? " PASS\n" : " FAIL\n");
    }
    std::cout << "\nPassed " << ok << " of " << n << '\n';

    std::cout << "\n---- original design on the new requirements -----\n";
    std::cout << "oldFare(Airport = (OldZone)3, peak) = " << oldFare(static_cast<OldZone>(3), true)
              << "  -> default branch: Airport journeys are denied\n";
    std::cout << "oldFare has no weekend parameter: weekend Outer peak-hours charges "
              << oldFare(OldZone::Outer, true) << " instead of 2700\n";
}
```

Output of the revised design (all 12 tests PASS)

```
---- fare tests ----
Airport  weekday  off-peak   expected  6000  actual  6000  PASS
Airport  weekday  peak       expected  7200  actual  7200  PASS
Airport  weekend peak hour   expected  5400  actual  5400  PASS
Outer  weekend  off-peak     expected  2700  actual  2700  PASS
Outer weekday peak           expected  3600  actual  3600  PASS
Middle  weekend  student     expected  1260  actual  1260  PASS
Inner  weekday  peak  senior  expected  840   actual  840   PASS
Invalid zone (Count)         expected -1    actual -1    PASS

---- monthly pass (Inner + Middle, valid days 1-30) ----
Inner  day 5 (covered)       expected 0     actual 0     balance 12000 PASS
Middle day 30 (covered)      expected 0     actual 0     balance 12000 PASS
Outer  day 5 (not in pass)   expected 3000  actual 3000  balance 9000  PASS
Inner  day 31 (expired)      expected 1000  actual 1000  balance 8000  PASS

Passed 12 of 12

---- original design on the new requirements ----
oldFare(Airport = (OldZone)3, peak) = -1  -> default branch: Airport journeys are denied
oldFare has no weekend parameter: weekend Outer peak-hours charges 3600 instead of 2700
```

D. Evidence-based defence (2 marks)

- **Test results:** 12 of 12 PASS: Airport weekday 6000 and peak 7200; Airport weekend in peak hours 5400 (discount replaces surcharge); Outer weekend 2700; Middle weekend student 1800 x 70 % = 1260; Inner weekday peak senior 1200 x 70 % = 840; invalid zone -1; pass journeys inside the pass cost 0, a journey outside the two zones costs 3000, and after the pass expires the normal 1000 is charged again.
- **The original design fails exactly where the evidence says:** Airport = -1 and the weekend Outer fare 3600 against the required 2700 (see part B).
- **A bug my own tests found:** in the first run the 'expired pass' test failed with -1 instead of 1000. The cause was my test data, not the rule: the card held exactly Rs 50 after the Outer journey, so the Rs 50 minimum correctly denied the next trip. I gave that card a larger starting balance (Rs 120) and the test passed. This shows the minimum-balance invariant still protects the card after the redesign.
- **Trade-offs:** a table is shorter to extend than a switch (one number plus the static_assert) but an invalid enum value would index outside the array, so validZone() is checked first; the switch had a safe default. The Context struct costs three bool parameters but makes the rules explicit and easy to extend (for example a holiday flag). The class keeps O(1) time per journey and 8 bytes of balance plus one pointer for the pass.
- **Assumptions affecting results:** weekend discount replaces the surcharge; discount then concession; pass covers 30 days and requires no balance check because nothing is deducted. If the metro decides otherwise, only the fare() function and one test line change - which is the point of keeping all rules in one place.

Final conclusion

The original design was sound for three zones and a weekday rule set, but needed MODIFICATION for the new requirements. The core ideas that the experiments supported - integer paise in long long, named constants in one place, enum class zones, a SmartCard that protects its own balance and table-driven tests - were all retained, and the changed requirements were added with small, testable changes.